

第一章 数字逻辑基础

十进制数	8421码 (有权码)	余3码 (无权码)	格雷码 (无权码)	余3循环码 (无权码)
0	0000	0011	0000	0010
1	0001	0100	0001	0110
2	0010	0101	0011	0111
3	0011	0110	0010	0101
4	0100	0111	0110	0100
5	0101	1000	0111	1100
6	0110	1001	0101	1101
7	0111	1010	0100	1111
8	1000	1011	1100	1110
9	1001	1100	1101	1010

• 将十进制数 (带小数) 转换为二进制数, 要求误差小于 2^{-N} , 则从整数点后写 N 位, 且最后一位四舍五入得到。

第二章 逻辑代数与硬件描述语言基础

最简表达式: ①最简与或式 ②最简与非-与非表达式 ③最简或与式 ④最简与非-或非式

• 分配律 $A+BC=(A+B)(A+C)$

吸收律 $A+\bar{A}B=A+B$

• 对偶规则: 将 L 中 "与" 或 "非" 互换, 得到 L 的对偶式 L'

• 与-或表达式: 先与后或, 或-与表达式: 先或后与

最小项: n 变量乘积项

最大项: n 变量或项

$m_i = \bar{M}_i$

第三章 逻辑门电路

• ODI: 普通 CMOS 门电路输出短路可能产生低阻通路, 大电流烧毁器件, 无源确定 (低、高电平)
ODI 门可线与, 工作时必须外接电源和电阻

上拉电阻计算: ①输出为 0 时, 至少一个 ODI 门漏电流 $I_{OL} \leq I_{OL(max)}$

$$\frac{V_{DD}-V_{OL(max)}}{R} + I_{IL(total)} \leq I_{OL(max)} \Rightarrow R \geq R_{min} = \frac{V_{DD}-V_{OL(max)}}{I_{OL(max)} - I_{IL(total)}}$$

②输出为 1 时, 全部 ODI 门漏电流 $I_{OH} \leq I_{OH(max)}$

$$\frac{V_{DD} - (I_{IH(total)} + I_{OH(total)})}{R} \geq V_{OH(min)} \Rightarrow R \leq R_{max} = \frac{V_{DD}-V_{OH(min)}}{I_{IH(total)} + I_{OH(total)}}$$

不管如何, 电流的输入 ODI 流入, 均由 V_{DD} 流出

• 重要参数: ①扇入数 N_i : 逻辑门输入端个数

②扇出数 N_o : 能带同类门电路输入端的最大数目

③拉电流 I_{OH} : 驱动门输出高电平时, 被拉出的电流

④灌电流 I_{OL} : 驱动门输出低电平时, 被灌入的电流

$$N_{OH} = \frac{I_{OH(max)}}{I_{IH}}$$

$$N_{OL} = \frac{I_{OL(max)}}{I_{IL}}$$

$$N_o = \min(N_{OH}, N_{OL})$$

• 实际问题:

(1) 门电路之间接口问题

逻辑门电路电平兼容: $V_{OH(min)} \geq V_{IH(min)}$, $V_{OL(max)} \leq V_{IL(max)}$

逻辑门电路电流兼容: $I_{OL(max)} \geq I_{IL(total)}$, $I_{OH(max)} \geq I_{IH(total)}$

(2) 带负载时接口电路: 驱动 LED 时, LED 工作电流 I_D 不大于 $I_{OH(max)}$ 和 $I_{OL(max)}$

(3) 多余输入端处理: 避免悬空以防干扰

第四章 组合逻辑电路

• 奇校验: 数据有奇数个 1 时, L 为 1, 否则为 0. 偶校验: 数据有偶数个 1 时, L 为 1, 否则为 0.

• 竞争: 逻辑门两输入端信号, 理论上同时发生反相变化, 而变化时间有差异, 由此产生错误脉冲的现象.
冒险: 由输出函数中的互变量引起的信号竞争, 在特定输入产生 $L=\bar{C}C$ 或 $L=CC$ 产生输出干扰脉冲的现象.

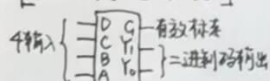
解决竞争冒险: ①卡诺图加冗余项 ②输出端并联电容器

• 编码器: 2^n 个输入, n 位输出

①普通编码器: 只允许输入一个有效信号, 否则输出无效码

②优先编码器: 允许同时输入多个有效信号, 输出其中优先级最高的码

[4-2线普通编码器]

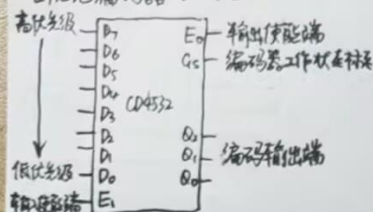


A	B	C	D	Y_1	Y_0	Q
1	0	0	0	1	1	1
0	1	0	0	1	0	1
0	0	1	0	0	1	1
0	0	0	1	0	0	1
其它				0	0	0

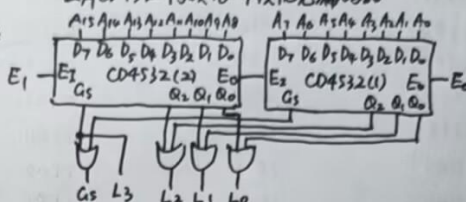
[4-2线优先编码器]

A	B	C	D	Y_1	Y_0	G
1	x	x	x	1	1	1
0	1	x	x	1	0	1
0	0	1	x	0	1	1
0	0	0	1	0	0	1
0	0	0	0	0	0	0

[优先编码器 CD4532]



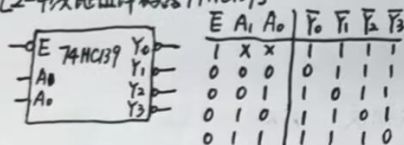
2片CD4532构成16-4线优先编码器



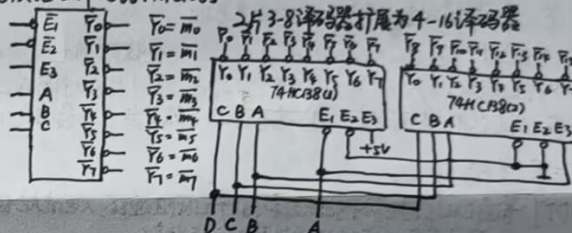
• 译码器: n 位输入, 2^n 个输出

①地址译码器 ②BCD译码器 ③显示译码器

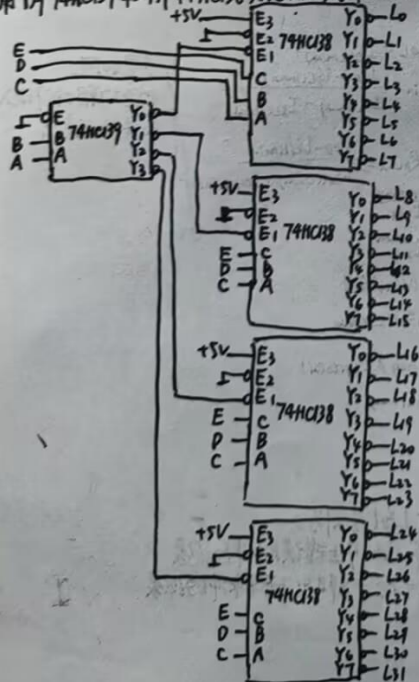
「2-4位地址译码器74HC139」



[3-8]线地址译码器74HC138



用1片74HC39和4片74HC138实现532线译码器



用74HC138实现逻辑函数 $L = \bar{A} \cdot \bar{C} + AB = m_0 + m_2 + m_6 + m_7 = \overline{m_1 m_3 m_4 m_5}$



用74HC138实现1-8线数据分配器

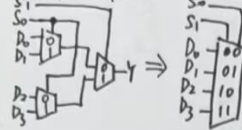
EN	E_2	E_3	E_1	A	B	C	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7
	5						D	1	1	1	1	1	1	1
D							D	0	1	1	1	1	1	1
							D	0	0	1	1	1	1	1
							D	0	1	0	1	1	1	1
							D	0	1	1	1	1	1	1
							D	1	0	0	1	1	1	1
							D	1	0	1	1	1	1	1
							D	1	1	1	1	1	1	1
							D	1	1	1	1	1	1	1
							D	1	1	1	1	1	1	1

数据选择器

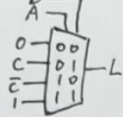
[2选1数据选择器]



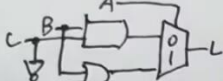
[4选1数据选择器]



4选1数据选择器实现 $L = AB + AC + BC = AB + A\bar{B}C + \bar{A}BC$



2选1数据选择器和逻辑门实现 $L(A, B, C) = \sum m(3, 4, 6, 7) = \bar{A}BC + A(B + \bar{C})$



数值比较器

[1位数值比较器]

A	B	G	L	E
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1

[2位数值比较器]

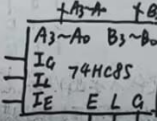


$$G = G_1 + E_1 G_0$$

$$E = E_1 E_0$$

$$L = L_1 + E_1 L_0$$

[四位数值比较器 74HC85]

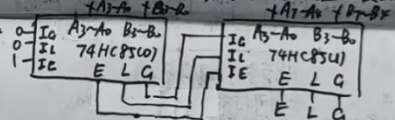


A ≠ B 时, 输出取决 A, B

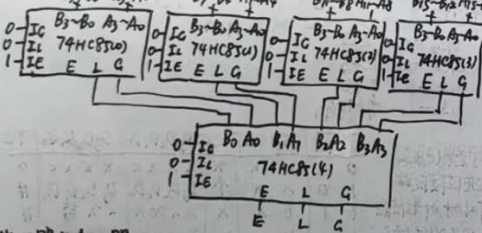
A = B 时, 输出取决 I_G, I_L, I_E

I _G	I _L	I _E	G	L	E
1	0	0	1	0	0
0	1	0	0	1	0
0	0	1	0	0	1
1	1	0	0	0	0
0	0	0	1	1	0

2片 74HC85 级联扩展实现 8 位数值比较器



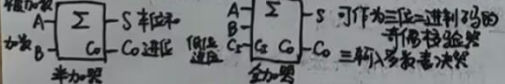
5片 74HC85 级联扩展实现 16 位数值比较器



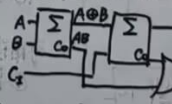
半加器与全加器

半加: 在两个 1 位二进制数相加时, 不考虑低位的进位的相加。

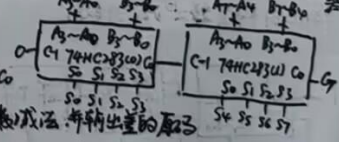
全加: 在两个 1 位二进制数相加时, 考虑低位进位的相加。



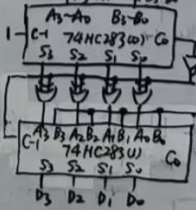
半加器构成全加器电路



用 2 片 74HC283 构成 7 8 位二进制加法器



用 74HC283 实现 4 位无符号数加法, 并输出进位的原理

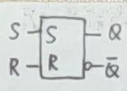


第五章 锁存器与触发器

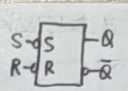
SR 锁存器

$$Q^{n+1} = R + \bar{Q}$$

$$\bar{Q}^{n+1} = S + Q$$

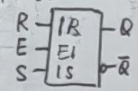


S	R	Q ^{next}	功能
0	0	Q	保持
0	1	0	置0
1	0	1	置1
1	1	X	不定

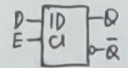


R	S	Q ^{next}	功能
1	1	Q	保持
0	1	0	置0
1	0	1	置1
0	0	X	不定

逻辑门实现SR锁存器



• D锁存器



[D锁存器 74HC373]

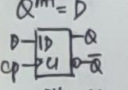
LE=1时数据输入, LE↓锁存
OE=0时输出

OE	LE	D	Q
0	1	1	1
0	1	0	0
0	0	X	Q ⁺
1	X	X	Z

• 触发器

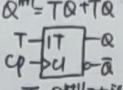
- ①主从触发器 ②维持阻塞触发器 ③利用传输延迟触发器

[D触发器]



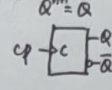
D	Q ^{next}	功能
0	0	置0
1	1	置1

[T触发器]



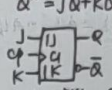
T	Q ^{next}	功能
0	Q	保持
1	Q ⁺	翻转

[T'触发器]



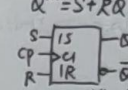
功能: 翻转

[JK触发器]



J	K	Q ^{next}	功能
0	0	Q	保持
0	1	0	置0
1	0	1	置1
1	1	Q ⁺	翻转

[SR触发器]



S	R	Q ^{next}	功能
0	0	Q	保持
0	1	0	置0
1	0	1	置1
1	1	X	不定

第2章 时序逻辑电路

- 检查自启动能力: 将电路无关状态依次代入求出次态, 若回到有效态, 则具有自启动能力
- 寄存器: 由触发器组成, 用于存储数据, 比锁存器有更好的同步干扰性 n位寄存器包含n个触发器, 可存储n位二进制代码

[8位寄存器 74HC374]

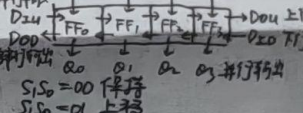
OE	CLK	D	Q
0	↑	1	1
0	↑	0	0
0	1	X	X
1	X	X	Z

[同步4位上移寄存器]



既有串行数据输出D₀
也有并行数据输出Q₀Q₁Q₂Q₃

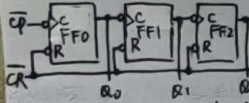
[多功能双向移位寄存器 74HC194]



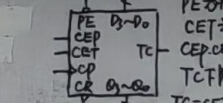
S₁S₀=00 保持
S₁S₀=01 上移
S₁S₀=10 下移
S₁S₀=11 并行输入

• 计数器: 既具有计数功能, 也具有分频功能

[异步模16计数器 74HC393]

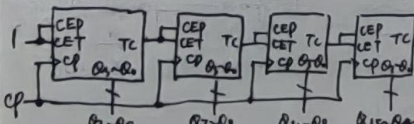


[同步模16计数器 74HC161]



CR	PE	CET	CET	CP	D ₃ D ₂ D ₁ D ₀	Q ₃ Q ₂ Q ₁ Q ₀	TC
0	X	X	X	X	X	X	0
1	0	X	X	↑	X	X	#
1	1	0	X	X	X	X	#
1	1	X	0	X	X	X	#
1	1	1	1	↑	X	X	#

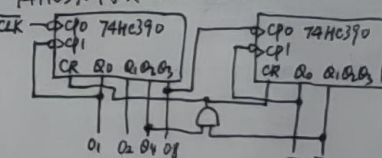
74HC161构成同步模16计数器



可用模M计数器实现模N计数器, 若M<N, 需用多片

- ①异步置0法 ②同步置数法

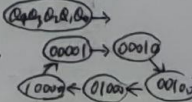
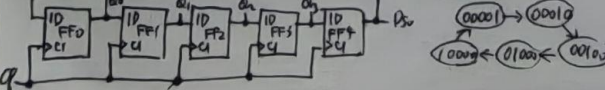
74HC390构成24进制计数器



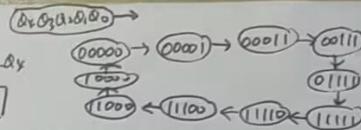
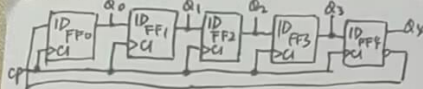
[异步多模(2/5/10)计数器 74HC390]

CP₀: 模2计数器 CP₁: 模5计数器
模2计数器状态输出接至模5计数器CP: 模10计数器, 8421码
模5计数器状态输出接至模2计数器CP: 模10计数器, 5421码

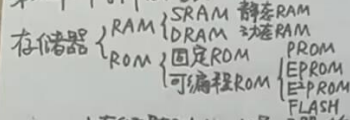
[不开分计数器]



扭环计数器 74HC4017



第七章 半导体存储器



- ROM: 由存储阵列、地址译码器、输出控制电路三部分组成 (字数) (字节)
- 字: 按位编组, 每次可读出一组 字长: 字的位长 字数: 字的总量 n根地址线产生2^n根字线, 输出数据位数为字长

[128Kx8位EPROM, AT27C010]

字长8, 地址线17根, 字数2^17=128K

ROM可实现组合逻辑电路功能

工作模式	CE	OE	PAM	A16-A0	V _{pp}	Q7-Q0
读	0	0	X	Ai	X	数据输出
输出无效	X	1	X	X	X	高阻
等待	1	X	X	Ai	X	高阻
快速编程	0	1	0	Ai	V _{pp}	数据输入
编程校验	0	0	1	Ai	V _{pp}	数据输出

ABC	F3 F2 F1 F0
0 0 0	1 0 0 1
0 0 1	0 0 1 0
0 1 0	0 0 1 0
0 1 1	0 1 1 1
1 0 0	0 0 1 0
1 0 1	0 1 1 1
1 1 0	0 1 1 1
1 1 1	1 1 0 0

F3: 一致性检测
F2: 多数表决
F1: 不一致检测
F0: 偶校验

- RAM: 随机读写, 掉电即失

① SRAM: 由存储阵列、地址译码、I/O控制三部分组成

工作模式	CE	WE	OE	I/O ₀ ~I/O _{n-1}
保持(低功耗)	1	X	X	高阻
读	0	1	0	数据输出
写	0	0	X	数据输入
输出无效	0	1	1	高阻

② SSRAM: 读写操作在CP同步控制下进行的

增加地址输入寄存器, 读写和从发控制逻辑电路
从发功能: 锁存首地址后, 在同步时钟作用下, 由内部计数器产生连续偏移地址
在连续读写多字时, 可减少外部地址总线占用时间, 提高读写效率

③ DRAM: 利用电容存储电荷效应, 电荷易失, 需定期刷新

读/写操作: 行地址分别送入地址寄存器后, 在使能信号控制下完成读写操作

刷新操作: 行地址不变, 只要列地址, 有效提高连续地址读写速度

行刷新操作: 一次刷新指定行的所有单元

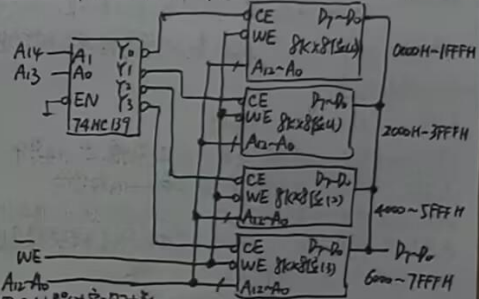
- 存储器容量扩展

① 字长扩展 (位扩展) ② 字数扩展

并联

增加高位地址, 经译码后产生各器件使能信号

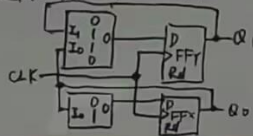
用8Kx8位的芯片组成2Kx8位的存储器



第八章 CPLD和FPGA

- CPLD: 内含多个逻辑块, 各块之间用可编程内部连线实现互连 采用GAL器件实现功能
- FPGA: 包括可编程逻辑块, 可编程互连开关, 可编程I/O接口 采用LUT实现功能

用可编程逻辑块实现2位二进制计数器

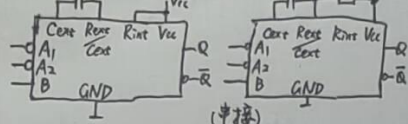


第九章 脉冲波形的变换与产生

- 微分型单稳态触发器 { 可重复触发
不可重复触发

$$t_w = RC \ln \frac{V_{DD}}{V_{DD} - V_{TH}} \quad f_{max} < \frac{1}{t_w + t_{re}}$$

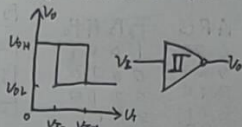
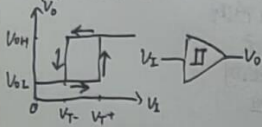
不可重复触发单稳态触发器 74121



功能: ①定时与测试 ②延时

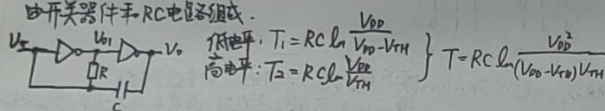
- 施密特触发器 { 同相输出 整形、抗干扰、波形变换
反相输出

电平触发器, 输入信号滞回时, 分别有正阈值 V_{T+} 和负阈值 V_{T-} .



- 多谐振荡器: 两个暂稳态, 无稳态.

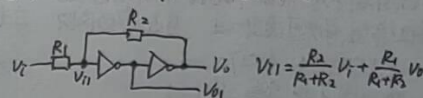
由开关器件和RC电路组成.



$$\left. \begin{array}{l} \text{低电平: } T_1 = RC \ln \frac{V_{DD}}{V_{DD} - V_{TH}} \\ \text{高电平: } T_2 = RC \ln \frac{V_{DD}}{V_{DD} - V_{TL}} \end{array} \right\} T = RC \ln \frac{V_{DD}}{(V_{DD} - V_{TH})(V_{DD} - V_{TL})}$$

$$\left. \begin{array}{l} \text{低电平: } T_1 = RC \ln \frac{V_{TH}}{V_{TH} - V_{TL}} \\ \text{高电平: } T_2 = RC \ln \frac{V_{DD} - V_{TH}}{V_{DD} - V_{TL}} \end{array} \right\} T = RC \ln \frac{V_{TH}(V_{DD} - V_{TH})}{V_{TH} - V_{TL}}$$

A1	A2	B	Q	Q̄
0	x	1	0	1
x	0	1	0	1
x	x	0	0	1
1	1	x	0	1
1	1	1	1	0
1	1	1	1	0
1	1	1	1	0
0	x	1	1	0
x	0	1	1	0
x	0	1	1	0



$$\begin{aligned} & \text{① } V_1=0, V_0=0, V_{TH}=0 \\ & V_1 \uparrow, \text{只要 } V_{TH} = V_{TH} \text{ 即 } \frac{R_2}{R_1+R_2} V_1 < V_{TH}, V_0=0 \text{ 发生转变} \\ & \Rightarrow V_{TH} = \left(1 + \frac{R_1}{R_2}\right) V_{TH} \\ & \text{② } V_1=1, V_0=1, V_{TH}=1 \\ & V_1 \downarrow, \text{只要 } V_{TH} = V_{TH} \text{ 即 } \frac{R_2}{R_1+R_2} V_1 > V_{TH}, V_0=1 \text{ 发生转变} \\ & \Rightarrow V_{TH} = \left(1 - \frac{R_1}{R_2}\right) V_{TH} \end{aligned}$$

第十章 数模和模数转换器

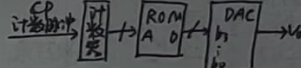
- DAC数模转换器

①权电阻网络 ②吉尔文串联网络 ③倒T形R-2R电阻网络 ④梯形R-2R电阻网络

输出方式: 单极性输出: 输入自然二进制数, 输出为正, 或为负
双极性输出: 输入有符号数, 输入数字量为正时, 输出为正; 输入数字量为负时, 输出为负.

$$\text{分辨率: } \frac{1}{2^n - 1}$$

应用: 数字式可编程增益控制电路, 脉冲产生电路, 波形发生.



- ADC模数转换器

取样 → 保持 → 量化 → 编码

[并行比较型ADC模数转换器] N位输出需 $2^N - 1$ 个器件

速度快, 电路复杂, 适用于速度快, 分辨率低的场合

[逐次比较型ADC模数转换器] n位输出转换一次需n个时钟周期

速度较快, 精度较高, 输出数字量位数越多, 转换精度越高. 位数越多, 时钟频率越高, 转换时间越短

[双积分型ADC模数转换器] $V_i = \frac{V_{REF}}{2^n} D$

抗干扰能力强, 避免非线性误差, 速度最慢

Verilog HDL 基础

带基数的表示方法: 表示常量 4'd8, 5'b03, 16'hE3-FF, 8'b1001-0011

变量 { 线网型 wire <[M:0]> 变量名;

寄存器型 reg <[M:0]> 变量名; 只可在 initial 或 always 语句块内被赋值。

[门级建模] 将逻辑电路图用 HDL 规定的文本语言表示出来

and 与门 or 或门 xor 异或门 rand 与非门 nor 或非门 xnor 同或门

buf 缓冲器 not 反相器

buf if 1 高电平有效控制三态缓冲器 not if 1 高电平有效控制三态反相器

buf if 0 低电平有效控制三态缓冲器 not if 0 低电平有效控制三态反相器

格式: and A1 (out, in1, in2, ...); not N1 (out1, out2, ..., in);

[模块] Verilog 描述电路的基本单元

```
module ABC (port1, port2, ...);  
    inout/input/output ...;  
    reg/wire/ ...;  
    parameter ...;  
  
    and/or/ ...;  
    assign ...;  
    always/initial  
    begin  
        if ... else ...;  
        case  
        ...  
    endcase  
    end  
endmodule
```

// 各模块通过端口相互连接
// 端口类型说明
// 变量类型说明
// 参数定义, 逗号分割多个参数
// 结构描述方式, 逻辑门或底层模块
// 数据流描述方式, 连续赋值语句
// 行为描述方式, 过程块结构
// 多条语句组成的行为描述块
// 条件描述语句
// 分支描述语句

[行为描述方式实现 2-1 选择器电路]

```
module mux2to1 (output reg Y, input [1:0] D, input S);  
    always @ (S, D)  $\rightarrow$  always 语句中被过程赋值的  
        if (S==1) Y=D[1];  
        else Y=D[0];  
endmodule
```

变量类型必须是寄存器型

[具有高电平使能的 3-8 线译码器行为建模]

```
module encoder3to8_bh (output reg [7:0] Y, input [2:0] A, input E);  
    integer k;  
    always @ (A, E)  
    begin  
        Y = 8'b11111111;  
        for (k=0; k<=7; k=k+1)  
            if ((E==1) && (A==k))  
                Y[k] = 0;  
        else  
            Y[k] = 1;  
    end  
endmodule
```

[条件运算符实现 2-1 数据选择器数据流行为建模]

```
module mux2to1_df (output L, input A, B, SEL);  
    assign L = SEL ? A : B;  
endmodule  
  
module mux2to1_df (output reg L, input A, B, SEL);  
    always @ (A, B, SEL)  
        L = SEL ? A : B;  
endmodule
```

[组合逻辑电路行为描述]

assign 结构: 连续赋值语句

always 结构: 过程赋值语句

if-else: 条件语句

case-endcase: 多路分支语句

for: 循环语句

[4 选 1 数据选择器行为建模]

```
module mux4to1 (output reg Y, input [3:0] D,  
                input C1, 0] S);  
    always @ (D, S)  
    if (S==2'b00) Y=D[0];  
    else if (S==2'b01) Y=D[1];  
    else if (S==2'b10) Y=D[2];  
    else Y=D[3];  
endmodule
```

[具有使能端口的 4 选 1 数据选择器行为建模]

```
module mux4to1_bh (output reg Y, input [3:0] D,  
                  input C1, 0] S, input En);  
    always @ (D, S, En)  
    begin  
        if (En==1) Y=0;  
        else  
            case (S)  
                2'd0: Y=D[0];  
                2'd1: Y=D[1];  
                2'd2: Y=D[2];  
                2'd3: Y=D[3];  
            endcase  
        end  
    end  
endmodule
```

[2-4 线译码器数据流建模]

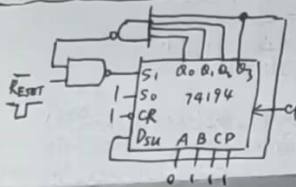
```
module decoder4_df (output [3:0] Y, input [1:0] A, input E);  
    assign Y[0] = ~(~A[0] & ~A[1] & ~E);  
    assign Y[1] = ~(~A[0] & A[1] & ~E);  
    assign Y[2] = ~(A[0] & ~A[1] & ~E);  
    assign Y[3] = ~(A[0] & A[1] & ~E);  
endmodule
```


[1位半加器, 1位全加器与4位全加器的实现]

```
module halfadder (output S, C, input A, B);
  xor g1 (S, A, B);
  and g2 (C, A, B);
endmodule
```

```
module fulladder (output S, CO, input A, B, C1);
  wire S1, D1, D2;
  halfadder HA1 (S1, D1, A, B);
  halfadder HA2 (S, D2, S1, C1);
  or g1 (CO, D1, D2);
endmodule
```

```
module _4bit_adder (output [3:0] S, output C3, input [3:0] A, B,
  input C1);
  wire CO, C1, C2;
  fulladder FA0 (S[0], CO, A[0], B[0], C1);
  FA1 (S[1], C1, A[1], B[1], CO);
  FA2 (S[2], C2, A[2], B[2], C1);
  FA3 (S[3], C3, A[3], B[3], C2);
endmodule
```

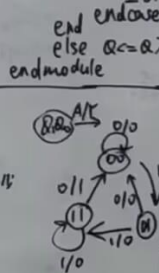


```
module shift194 (output reg [3:0] Q, input [3:0] S, input Dsd, Dsu, CP, CR, input [3:0] D);
  always @ (posedge CP, negedge CR)
    if (~CR) Q <= 4'b0000;
    else
      case (S)
        2'b00: Q <= Q;
        2'b01: Q <= {Q[2:0], Dsu};
        2'b10: Q <= {Dsd, Q[3:1]};
        2'b11: Q <= D;
      endcase
endmodule
```

[74HC161 同步模16计数器]

```
module counter161 (output reg [3:0] Q, output TC, input CP, CR, PE, CE, CET, input [3:0] D);
  wire CE;
  assign CE = CEP & CET;
  assign TC = CET & (Q == 4'hf);
  always @ (posedge CP, negedge CR)
    if (~CR) Q <= 4'h0;
    else if (~PE) Q <= D;
    else if (~CE) Q <= Q;
    else Q <= Q + 1'b1;
endmodule
```

```
module moore_machine
  (output reg [1:0] Q,
   input A, CP, CR);
  always @ (posedge CP, negedge CR)
    begin
      if (~CR) Q <= 2'b00;
      else
        case (A)
          2'b00: if (~A) Q <= 2'b01;
          2'b01: if (A) Q <= 2'b10; else Q <= 2'b11;
          2'b10: if (~A) Q <= 2'b11;
          2'b11: if (~A) Q <= 2'b00;
        endcase
    end
endmodule
```



[锁存器和触发器的行为描述]

always 语句块: 描述事件发生时, 电路的行为过程。
电平敏感事件: always @ (EN, CR)
边沿敏感事件: always @ (posedge CP, negedge CR)

```
module D_latch (output reg Q, input D, E);
  always @ (E, D)
    if (E) Q <= D;
endmodule
```

```
module DFF (output reg Q, input D, CP);
  always @ (posedge CP)
    Q <= D;
endmodule
```

[D触发器的异步与同步置0行为]
异步置0: module A_DFF (output reg Q, input D, CP, RD);
always @ (posedge CP, negedge RD)
Q <= (~RD) ? 1'b0: D;
endmodule
同步置0: module S_DFF (output reg Q, input D, CP, RD);
always @ (posedge CP)
Q <= (~RD) ? 1'b0: D;
endmodule

```
module 4bit_counter (output reg [3:0] Q,
  input CP, CLR, Mod);
  always @ (posedge CP, negedge CLR)
    if (~CLR) Q <= 4'b0000;
    else Q <= (Mod ? Q + 1'b1: 1'b0);
endmodule
```

改为 0 + (-1) 更合适
无溢出移位寄存器

S1	S0	计数
0	0	0000
0	1	0001
1	0	0010
1	1	0011

```
module mealy_sequence_detector (output reg Y,
  input A, CP, CR);
  reg [0] state, next_state;
  always @ (negedge CP, negedge CR) begin
    if (~CR) state = 2'd0;
    else state = next_state;
  end
  always @ (state, A) begin
    case (state)
      2'd0: begin Y = 1'b0; next_state = (A == 1) ? 2'd1: 2'd0; end
      2'd1: begin Y = 1'b0; next_state = 2'd3; end
      2'd3: begin Y = 1'b0; next_state = 2'd0; end
      default: begin Y = 1'b0; next_state = 2'd0; end
    endcase
  end
endmodule
```